

我把论文和作者公开的 GitHub 仓库都对了一遍。先给你一个最重要的定位：

DelTA 不是“另一个 GRPO advantage estimator”，而是在 GRPO/DAPO 这类 sequence-level RLVR 上增加了一层 token-level credit assignment。

GRPO/DAPO 主要回答：**这一整条 response 应该被奖励还是惩罚多少？** DelTA 进一步回答：**这条 response 里面，哪些 token 的梯度更值得学习，哪些 token 虽然属于正确/错误 response，但其实是公共噪声，应该少学一点？** 而且代码里这一点非常明确：训练脚本仍然设置 `adv_estimator=grpo`，DelTA 是在得到 group-relative advantage 之后，再生成 token-wise 权重。 [GitHub](#)

1. 这篇论文到底在解决什么问题？

论文是 **DelTA: Discriminative Token Credit Assignment for Reinforcement Learning from Verifiable Rewards**。作者从一个 RLVR 中很基础、但容易被忽略的问题出发：假设模型生成：

```
Response A:
Let's think step by step.
...
因此答案是 42。
Reward = 1

Response B:
Let's think step by step.
...
因此答案是 37。
Reward = 0
```

GRPO 一类方法通常先计算 response-level advantage：

$$\hat{A}_i = \frac{R_i - \mu_R}{\sigma_R + \epsilon}$$

于是同一个 response 里的所有 token 都共享一个 $\hat{A}_i$ 。论文明确指出，这产生了一个 **granularity mismatch**：reward 是 response-level 的一个标量，但真正做梯度更新时，却是一个 token 一个 token 累积梯度。例如正确 response：

```
Let's      A = +1
think      A = +1
step       A = +1
by         A = +1
step       A = +1
...
42         A = +1
```

如果只从 reward 看：

这些 token 都是“正确 response 中的 token”。但它们显然不是同等重要。Le
t's、step、换行、标点等，在正确和错误回答中都大量出现；真正区分正确和错误推理的，也许只有很少几个关键 reasoning token。这正是 Delta 要解决的 **Token Credit Assignment**。

2. 论文最有意思的地方：把 RLVR 看成一个“分类器”

这是这篇论文理论部分最值得理解的地方。设某个 token 的梯度向量：

$$v_{i,t} = \nabla_{\theta} \log \pi_{\theta}(o_{i,t} | q, o_{i,<t})$$

那么对于一次很小的参数更新 $\Delta\theta$ ，Taylor 展开：

$$\Delta \log \pi(x|c) \approx \nabla_{\theta} \log \pi(x|c)^T \Delta\theta$$

也就是说：

某个 token 概率最终升还是降，取决于它的 **gradient vector** 与总更新方向 $\Delta\theta$ 的**内积**。对于普通 sequence-level RLVR，可以把更新写成：

$$\Delta\theta_{\text{RLVR}} \propto \sum_{\hat{A}_i > 0} \sum_t \hat{A}_i v_{i,t} - \sum_{\hat{A}_i < 0} \sum_t |\hat{A}_i| v_{i,t}$$

于是把正 advantage token 放到一起，可以得到一个正类 “中心”：

$$\bar{\mu}_+ = \frac{\sum_{\hat{A}_i > 0, t} \hat{A}_i v_{i,t}}{M_+}$$

负 advantage token 同理：

$$\bar{\mu}_- = \frac{\sum_{\hat{A}_i < 0, t} |\hat{A}_i| v_{i,t}}{M_-}$$

最终：

$$\Delta\theta \propto M_+ \bar{\mu}_+ - M_- \bar{\mu}_-$$

论文因此得到一个很巧妙的解释：

RLVR 的 policy-gradient update，本身等价于在 token-gradient space 里构造了一个隐式线性 discriminator。 对于候选 token：

- 梯度更像 μ_+ → 倾向提高概率；
- 梯度更像 μ_- → 倾向降低概率。

3. 普通 GRPO/DAPO 的问题出在哪里？

问题就在这个 centroid。假设正负两边的 token-gradient 是：



Negative responses

```
"Let's"      •
"think"     •
"step"       •
"37"         •
```

大量公共 token:

```
Let's
think
step
...
,
.
newline
```

既存在于正确回答，也存在于错误回答。而 centroid 本质上是平均:

```
positive:
 $\mu_+$  = average(all positive tokens)

negative:
 $\mu_-$  = average(all negative tokens)
```

所以大量高频公共 token 会把两个 centroid 都拉向:

```
shared region
↓
 $\mu_-$  ← ●●●● →  $\mu_+$ 
```

导致:

$$\mu_+ - \mu_-$$

反而不够“判别”。论文把这称为：

好的 **within-side summary** 不一定是好的 **between-side discriminator**。这也是 Figure 1 想表达的核心：一开始正负两类 gradient cloud 有大量重叠的 shared features；DelTA 通过重新赋权，把 centroid 从公共区域拉向真正有区别的 gradient direction。

4. DelTA 做了什么？

核心实际上就一句话：

给每一个 token 再乘一个 $\lambda_{i,t}$ 。普通 DAPO：

$$\hat{A}_i$$

DelTA：

$$\lambda_{i,t}\hat{A}_i$$

因此同一条 response 不再：

Token	Advantage
Let's	+1
think	+1
step	+1
42	+1

而可能变成：

Token	A_i	$\lambda_{i,t}$	effective weight
-------	-------	-----------------	------------------

Let's	+1	0.82	0.82
think	+1	0.87	0.87
step	+1	0.83	0.83
...			
关键推理词	+1	1.16	1.16
42	+1	1.19	1.19

注意论文默认的范围非常保守：

$$\lambda \in [0.8, 1.2]$$

它不是直接把“不重要 token”删掉，而是做轻量 reweighting。主实验使用 $K = 1$ 。

5. 那 $\lambda_{i,t}$ 怎么算？

这是 DelTA 最核心的算法。先有：

$$\mu_+, \quad \mu_-$$

对于一个 **positive-response token**，算它到两个中心的距离：

$$d_+ = \|v_{i,t} - \mu_+\|^2$$

$$d_- = \|v_{i,t} - \mu_-\|^2$$

然后看：

$$\Delta_+ = d_- - d_+$$

如果：

$$d_- \gg d_+$$

说明这个 token：

很像 positive tokens，明显不像 negative tokens。那么它很有判别力。反过来，如果：

$$d_- \approx d_+$$

它两边都像，很可能就是：

```

Let's
therefore
,
.
\n

```

之类的 shared feature。论文把它通过 sigmoid 映射：

$$\alpha_{i,t} = \sigma\left(\frac{\|v - \mu_- \|^2 - \|v - \mu_+ \|^2}{\gamma_+}\right)$$

对于 negative token 则反过来：

$$\alpha_{i,t} = \sigma\left(\frac{\|v - \mu_+ \|^2 - \|v - \mu_- \|^2}{\gamma_-}\right)$$

最后：

$$\lambda_{i,t} = \lambda_{\min} + (\lambda_{\max} - \lambda_{\min})\alpha_{i,t}$$

所以：

```

自己这一类明显特有
↓
alpha → 1
↓
lambda → 1.2

```

而：

```


```

正负两边都差不多

↓

alpha $\rightarrow$ 0.5

↓

lambda $\rightarrow$ 1.0

甚至更像 opposite side:



alpha $\rightarrow$ 0

lambda $\rightarrow$ 0.8

6. 它还不是只算一次 centroid

作者发现:



原始 μ^+

原始 μ^-

本身已经被 shared token 污染。所以进一步迭代:



原始 centroid

↓

算每个 token 的 alpha

↓

高判别 token 权重大

↓

重新计算 centroid

↓

新的 μ^+ / μ^-

↓

重新计算 token 权重

更新:

$$\mu_+^{(k+1)} = \frac{\sum_{\hat{A}_i > 0, t} \hat{A}_i \alpha_{i,t}^{(k)} v_{i,t}}{\sum_{\hat{A}_i > 0, t} \hat{A}_i \alpha_{i,t}^{(k)}}$$

负侧对称。主实验只：

$$K = 1$$

也就是 refine 一轮。消融实验很有意思：去掉 refinement 后，所选四个 benchmark 的平均分从 **23.27** 降到 **19.97**，是几个组件中下降最大的。

7. 这里有一个工程上非常关键的问题：不可能真的算完整 token gradient

理论上：

$$v_{i,t} = \nabla_{\theta} \log \pi(y_t)$$

假设 8B 模型，一条 response 20K token。你显然不可能保存：

```
[B, S, 8 billion parameters]
```

的梯度。论文因此用了一个非常漂亮的近似。最后一层：

$$z_t = Wh_t$$

对实际输出 token y_t 对应 LM-head row 求导：

$$\nabla_{W_{y_t}} \log p_t(y_t) = (1 - p_t(y_t))h_t$$

所以直接把：

$$v_t = (1 - p_t(y_t))h_t$$

当做 token-gradient proxy。论文明确强调：

这不是 full-parameter gradient 的精确重建，只是用于计算 stop-gradient token coefficient 的 last-layer proxy。这个地方和代码完全一致。作者代码：

```
probs = torch.exp(log_probs)
v = (1.0 - probs).unsqueeze(-1) * h
```

实际 tensor:

```
log_probs : [B, S]
h          : [B, S, H]

          ↓

v          : [B, S, H]
```

[GitHub](#) 这算是论文公式 → 源码最直接的——对应。

8. 代码仓库实际上是怎么改 verl 的？

仓库是公开的 [RUCBM/Delta](#)，作者明确说明代码主要基于 [verl](#)，训练入口是：

```
verl-Delta/recipe/dapo/srcs/run_Delta.sh
```

[GitHub](#) 最值得注意的是启动参数：

```
adv_estimator=grpo

loss_agg_mode="token-mean"

clip_ratio_low=0.2
clip_ratio_high=0.28
```

```
delta_K=1
delta_lam_min=0.8
delta_lam_max=1.2
delta_impl=Normal
```

而且：

```
use_kl_in_reward=False
use_kl_loss=False
```

[GitHub](#) 这直接说明了 DelTA 和 GRPO 的关系：

```
GRPO advantage estimator
    ↓
    A_i
    ↓
    DAPO-style clipped/token-mean policy loss
    ↓
    DelTA 再计算  $\lambda_{i,t}$ 
    ↓
    effective token advantage
     $\lambda_{i,t} \times A_i$ 
```

所以不能把它理解成：

```
GRPO vs DelTA
```

这种完全平行的关系。 更准确应该是：

```
Group-relative RLVR
    |
    ▼
    GRPO advantage
    |
    └──┬──┘
```



论文自己也把 **DAPO** 作为主要 concrete baseline，而不是 vanilla GRPO。DAPO 相比 GRPO 又加入 asymmetric clipping 和 token-level loss normalization。

9. GRPO 和 DeITA 最本质的区别

论文 Appendix E 其实专门把 GRPO 写得非常清楚。GRPO 的局部更新：

$$\Delta\theta_{\text{GRPO}} \propto \frac{1}{G} \sum_i \frac{\hat{A}_i}{|o_i|} \sum_t v_{i,t}$$

也就是说：

$$\rho_{i,t}^{\text{GRPO}} = \frac{1}{|o_i|}$$

同一个 response 中的所有 token：

```
token 1    1/L
token 2    1/L
token 3    1/L
...
token L    1/L
```

完全一样。换句话说，GRPO 是：

$$\bar{v}_i = \frac{1}{|o_i|} \sum_t v_{i,t}$$

先把整条 response 的 token gradient 平均，再乘 response advantage。DeITA 则是：

$$\rho_{i,t}^{DelTA} = \lambda_{i,t}$$

每一个 token 都不同。

维度	GRPO	DelTA
Reward	response-level	response-level
Advantage	group relative $\hat{A}_i$	仍然可用 GRPO 的 $\hat{A}_i$
同一 response 内 advantage	一样	基础 A_i 一样
token 权重	(1/	o_i
Credit assignment	主要 response-level	显式 token-level reweight
判断依据	response reward	token-gradient 与正负 centroid 的距离
Critic	不需要	不需要
Process Reward Model	不需要	不需要
额外模型	不需要	不需要
额外计算	较少	centroid/proxy 的额外 forward

所以一句话可以记成：

GRPO: 哪个 response 更好？

而：

DelTA: 这个 response 里，到底哪些 token 真正导致了“好/坏”？

不过更严谨一点，DelTA 并不是推断因果意义上的“哪个 token 导致 reward”，而是在问：

哪些 token-gradient direction 最能区分 positive-advantage 和 negative-advantage responses。这是二者非常重要的差别。

10. 源码完整的数据流

核心函数在：

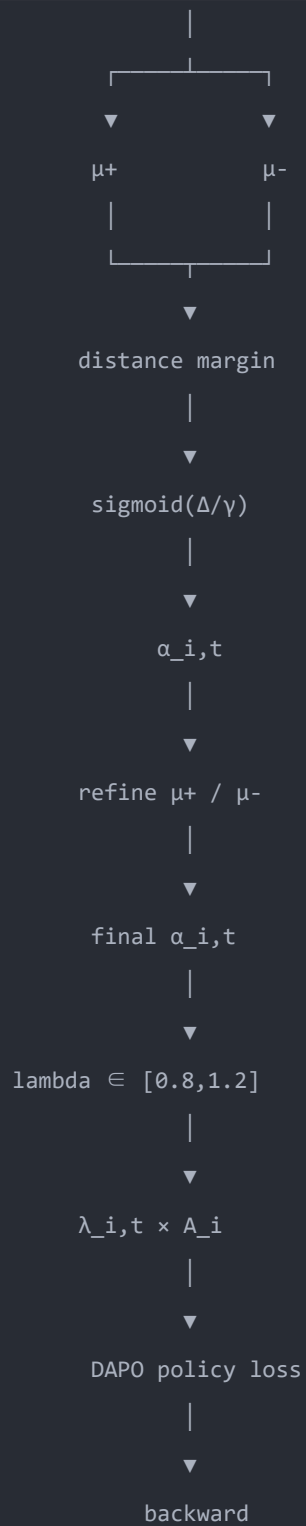
```
DataParallelPPOActor.compute_log_prob_h(...)
```

它返回的不只是 log prob, 还包括：

```
log_probs
W
b
preds
weights
weighted_advantages
```

[GitHub](#) 代码流程基本可以还原成：

```
rollout
  |
  ▼
reward
  |
  ▼
GRPO advantage
A_i
  |
  ▼
forward actor
  |
  ├── log_prob
  |
  └── final hidden state h
      |
      ▼
      v=(1-p(y))*h
      |
      ▼
      positive / negative
      token gradient
```



注意前面的：



```
v = (1.0 - probs).unsqueeze(-1) * h
```

以及 refinement:



```
margin_pos = d_neg - d_pos
margin_neg = d_pos - d_neg

raw_w_pos = torch.sigmoid(margin_pos / gamma_pos)
raw_w_neg = torch.sigmoid(margin_neg / gamma_neg)
```

然后 centroid:

```
w_pos = raw_w_pos * positive_advantage * valid_mask

num_pos += (w_pos.unsqueeze(-1) * v).sum(...)
den_pos += w_pos.sum()
```

这几乎就是论文 Eq. 6 + Eq. 7 的直接翻译。 [GitHub](#)

11. γ 也不是随便定的

论文没有把 temperature 写死。它先统计:

$$\Delta = d_- - d_+$$

在 positive 和 negative token 上的方差, 然后:

$$\gamma_+ = \sqrt{\max(\text{Var}(\Delta_+), \epsilon)}$$

negative 同理。所以本质是在做:

```
raw distance margin
    ↓
除以该 side margin 的自然尺度
    ↓
sigmoid
```

避免某一 batch 中距离绝对值特别大, 导致 sigmoid 全饱和。

12. Self-normalization 在代码里是怎么实现的？

这一块特别值得看。论文最终目标看起来是：

$$J_{\text{DelTA}} = \frac{\sum_{i,t} \lambda_{i,t} \ell_{i,t}}{\sum_{i,t} \lambda_{i,t}}$$

但 verl 原来的 DAPO 是 token mean：

$$\frac{1}{N} \sum_{i,t} \ell_{i,t}$$

为了不改整个 loss aggregation，作者定义：

$$\bar{\lambda}_{i,t} = \lambda_{i,t} \frac{N}{\sum \lambda}$$

于是：

$$\frac{1}{N} \sum \bar{\lambda}_{i,t} \ell_{i,t} = \frac{\sum \lambda_{i,t} \ell_{i,t}}{\sum \lambda_{i,t}}$$

论文在 Appendix H 把这个实现技巧专门写出来了。代码对应：

```
gy_norm = tk_sum / wht_sum.clamp_min(1e-8)

weights *= gy_norm
weighted_advantages *= gy_norm
```

[GitHub](#) 也就是说最终：

```
mean(lambda_bar) = 1
```

因此不会因为：

```
一批 token 平均  $\lambda = 1.13$ 
```

就把整个学习率隐式放大 13%。这是很重要的设计。

13. 源码里还有一个值得注意的小细节

代码中有：

```
d = sig_pos + sig_neg + lam
d = torch.ones_like(mu_pos)
```

也就是说前一行刚构造了某种按维度 scale 的 `d`，下一行直接覆盖成了全 1：

```
d = 1
```

随后距离实际计算：

```
d_pos = ((diff_pos ** 2) / d).sum(...)
```

因此公开代码实际最终使用的还是普通的 squared Euclidean distance：

$$\|v - \mu\|_2^2$$

与论文正文描述一致。[GitHub](#) 从代码阅读的角度看，前面的 `sig_pos + sig_neg + lam` 更像是开发过程中尝试过的按维度 variance scaling 残留，但**当前公开实现并没有真正启用它**。这是我根据代码做的推断。

14. 为什么要有 Normal 和 Memory_efficient 两个实现？

训练脚本中：

```
delta_impl=Normal
```

作者注释得很清楚：

```
Normal:
Direct implementation.
Easier to read/debug, but uses more GPU memory.

Memory_efficient:
Algorithmically equivalent,
avoids several large [B,S,H] intermediate tensors.
```

并说明大多数实验用 `Normal`，Qwen3-14B-Base 使用 memory-efficient 版本。[GitHub](#) 原因很好理解。因为：

$$v : [B, S, H]$$

直接算：

```
diff = v - mu
distance = (diff ** 2).sum(-1)
```

会产生大的：

```
[B,S,H]
```

临时 tensor。Memory-efficient 版本把：

$$\|v - \mu\|^2$$

展开为：

$$v^T v - 2v^T \mu + \mu^T \mu$$

源码确实这么干：



```
v2_d_sum = ((v ** 2) / d).sum(dim=-1)

mu_pos_d = mu_pos_star / d

v_mu_pos = torch.matmul(v, mu_pos_d)
```

来避免重复 materialize 大型 `[B,S,H]` 中间张量。 [GitHub](#)

15. DelTA 的代价有多大？

这并不是“免费”的。论文指出，如果把所有 rollout hidden states 缓存下来，理论上 refinement 不需要重新 forward；但长 CoT 下缓存全部 hidden state 太吃显存，所以实际实现选择重新 forward。对于 K 次 refinement：

$$K + 2$$

次额外 actor forward 用来估计 coefficient。主实验：

$$K = 1$$

所以相当于多 **3 次 proxy-related actor forward**。在 8×B200 上，作者测得第一步比 DAPO 多约 **37 秒**，约占 DelTA first-step 时间的 **10.2%**。所以它的 trade-off 很清楚：

GRPO/DAPO

↓

计算简单

DelTA

↓

多几次 forward

↓

换更精细的 token credit

16. 效果是不是明显？

主表没有拿 vanilla GRPO 做主要 baseline，而是拿更强的 DAPO / SAPO / FIPO 等比较，这一点需要特别强调，不能把论文结果说成“DeITA 比 GRPO +3.26”。8B：

Method	Avg
DAPO	22.95
DAPO + FT	24.80
SAPO	25.14
FIPO	23.89
DelTA	28.40

14B：

Method	Avg
DAPO	35.09
DAPO + FT	36.77
SAPO	35.94
FIPO	37.29
DelTA	39.91

所以作者所谓的：

- 8B：+3.26
- 14B：+2.62 是相对于最强 same-scale baseline。

17. 我觉得最能证明 DeITA 不是“花哨 reweight”的实验

不是主表，而是 Top-50% / Bottom-50% token 实验。作者计算 λ 后：

Top 50% λ token
Random 50%
Bottom 50% λ token
Full DAPO

结果：

只训练 Top-50% token, 反而能超过 full-token DAPO; 而: Bottom-50% token 训练会迅速 collapse。论文因此认为低- λ token 不只是“无用”, 它们的一部分 gradient direction 甚至会妨碍 RLVR update。这个实验其实比“accuracy +3 分”更支持它的 token-credit hypothesis。

18. 还有一个非常关键的消融: 只看 own-side 为什么不行?

你可能会想:

那我不管 negative centroid, 只看 token 离自己 μ_+ 近不近不就完了? 作者试了。结果:



```
DelTA          23.27
DAPO           19.05
Within-side-only 17.94
```

甚至 **比 DAPO 还差**。这反而证明了论文标题中的 **Discriminative** 是关键: 不是:

这个 token 是不是“典型的正确回答 token”

而是:

它是不是比错误回答更像正确回答

例如:



```
"Let's"
```

可能非常靠近 positive centroid。但:



```
"Let's"
```

也非常靠近 negative centroid。因此它：



```
own-side centrality: 高  
discriminative power: 低
```

DeITA 后者才是真正想抓的。

19. 把 GRPO 和 DeITA 写成代码，你一眼就能看出来区别

GRPO 可以极度简化成：



```
# 一个 prompt 采样 G 个 response  
responses = rollout(policy, prompt, G)  
  
rewards = reward(responses)  
  
advantages = normalize_group(rewards)  
  
for i, response in enumerate(responses):  
    for t in response:  
        loss[t] = -advantages[i] * ratio[i, t]  
  
loss = response_mean_or_grpo_aggregation(loss)  
loss.backward()
```

同一 response：



```
advantages[i]
```

对所有 token 都一样。DeITA 则相当于：



```

responses = rollout(policy, prompt, G)
rewards = reward(responses)

advantages = normalize_group(rewards)

# ---- DeLTA 新增 ----

log_probs, h = actor_forward(responses)

v = (1 - exp(log_probs))[..., None] * h

mu_pos = weighted_centroid(
    v,
    weight=positive_advantage
)

mu_neg = weighted_centroid(
    v,
    weight=negative_advantage
)

for k in range(K):

    d_pos = squared_distance(v, mu_pos)
    d_neg = squared_distance(v, mu_neg)

    alpha_pos = sigmoid(
        (d_neg - d_pos) / gamma_pos
    )

    alpha_neg = sigmoid(
        (d_pos - d_neg) / gamma_neg
    )

    mu_pos = weighted_centroid(
        v,
        positive_advantage * alpha_pos
    )

    mu_neg = weighted_centroid(
        v,
        negative_advantage * alpha_neg

```



```

    )

# 最终 weight
alpha = discriminative_score(v, mu_pos, mu_neg)

lambda_ = 0.8 + 0.4 * alpha

# self normalization
lambda_bar = lambda_ * num_tokens / lambda_.sum()

# ---- RL Loss ----

effective_advantage = (
    advantages[:, None] * lambda_bar
)

loss = dapo_loss(
    ratio,
    effective_advantage
)

loss.backward()

```

这和公开代码的数据流基本一致：源码确实使用 GRPO advantage，然后用 final hidden state 构建 $v = (1 - p)h$ ，计算正负 centroid、margin、sigmoid coefficient，再做 global coefficient normalization。 [GitHub](#)

最后把它压缩成一句最准确的话

GRPO 的 credit assignment 基本停留在 response 层：

$$R_i \rightarrow A_i \rightarrow \text{response 内 token 共享}$$

DeITA 在其上增加 token 层的 discrimination：

$$R_i \rightarrow A_i \rightarrow v_{i,t} \rightarrow (\mu_+, \mu_-) \rightarrow \lambda_{i,t} \rightarrow \boxed{\lambda_{i,t} A_i}$$

所以我更愿意把 DeITA 定义成：

“GRPO/DAPO 上的 gradient-space contrastive token reweighting”，而不是把它说成一个和 GRPO 完全并列的新 RL 算法。另外，从源码看，这篇工作最核心的改动其实相当集中：**GRPO advantage 没动、reward 没动、rollout 没动，主要新增的是 `compute_log_prob_h()` 这一整套 token-gradient proxy → centroid refinement → weighted advantage 计算。**这也是为什么它非常适合直接嵌到现有 verl/DAPO pipeline 里。 [GitHub](#)